

Experiment – 1

Term Frequency, Tokenization, Stop Word Removal and Stemming using Python

Name : Ankna Litoriya

Github repo : <https://github.com/Ankna-litoriya/M.Tech>

1. Aim

To perform **Tokenization, Stop Word Removal, and Stemming** on technical text and to calculate the **Term Frequency (TF)** of selected technical terms using Python and the Natural Language Toolkit (NLTK).

2. Objective

- To understand the concept of **Natural Language Processing (NLP)**.
 - To perform **tokenization of technical text**.
 - To remove **stop words from the dataset**.
 - To apply **stemming to reduce words to their root form**.
 - To calculate and **rank technical terms based on term frequency**.
-

3. Theory

Natural Language Processing (NLP)

Natural Language Processing is a branch of artificial intelligence that focuses on enabling computers to understand, interpret, and process human language. NLP techniques are widely used in text mining, information retrieval, sentiment analysis, and chatbots.

Tokenization

Tokenization is the process of **breaking a large text into smaller units called tokens**. These tokens may be words, phrases, or sentences. Tokenization is the first step in text preprocessing.

Example:

Text → *Big data processing is important*

Tokens →

Big, data, processing, is, important

Stop Words

Stop words are **commonly used words that carry little meaning** in text analysis. Examples include *is, the, and, in, on*. Removing stop words helps improve processing efficiency and focuses on meaningful words.

Stemming

Stemming is the process of **reducing words to their root form**. It removes suffixes or prefixes.

Example:

Word	Stem
processin g	process
computing	comput
analytics	analyt

Stemming helps reduce vocabulary size in NLP tasks.

Term Frequency (TF)

Term Frequency measures **how often a word appears in a document**.

Formula:

$TF(t) = (\text{Number of times term } t \text{ appears in the document})$

Higher frequency indicates that the word is more important in the document.

4. Software Requirements

- Python 3.x
 - Anaconda Navigator / Jupyter Notebook
 - NLTK Library
 - Pandas Library
 - Python-docx Library
-

5. Algorithm

1. Import required Python libraries.
 2. Download required NLTK datasets.
 3. Upload and read the technical content from the Word file.
 4. Perform tokenization on the text.
 5. Remove stop words from the tokenized words.
 6. Apply stemming to at least 10 words.
 7. Select 4–5 technical terms.
 8. Calculate the term frequency for each term.
 9. Rank the technical terms according to their frequency.
 10. Display the results.
-

6. Program

```
import nltk
import pandas as pd
from docx import Document
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')

file_path = "big_data_technical_content_3_pages.docx"

doc = Document(file_path)

text = ""

for para in doc.paragraphs:
    text += para.text + " "

tokens = word_tokenize(text)

print("Tokenized Words:")
print(tokens[:50])

stop_words = set(stopwords.words('english'))

filtered_tokens = [word.lower() for word in tokens if word.lower() not in stop_words and
word.isalpha()]

print("\nWords After Stopword Removal:")
print(filtered_tokens[:50])

stemmer = PorterStemmer()

sample_words = filtered_tokens[:10]

stemmed_words = [stemmer.stem(word) for word in sample_words]

print("\nOriginal Words:", sample_words)
print("Stemmed Words:", stemmed_words)

technical_terms = ["data", "hadoop", "spark", "kafka", "processing"]

term_frequency = {}
```

```
for term in technical_terms:
    term_frequency[term] = filtered_tokens.count(term)

print("\nTerm Frequency:")
print(term_frequency)

ranked_terms = sorted(term_frequency.items(), key=lambda x: x[1], reverse=True)

print("\nRanked Technical Terms:")

for rank,(term,freq) in enumerate(ranked_terms,start=1):
    print(rank, term, freq)

df = pd.DataFrame(ranked_terms, columns=["Technical Term","Frequency"])

df["Rank"] = df["Frequency"].rank(ascending=False, method="dense").astype(int)

print("\nRanking Table")
print(df)
```

7. Output

Tokenized Words

Example output:

Big, Data, refers, to, extremely, large, datasets, used, for, processing, etc.

Words After Stop Word Removal

Big, Data, refers, extremely, large, datasets, processing, distributed, computing, framework, etc.

Stemming Result

Original Word	Stemmed Word
processing	process
computing	comput

analytics	analyt
learning	learn
systems	system

Term Frequency Table

Technical Term	Frequency
data	15
spark	6
hadoop	5
kafka	4
processing	3

Ranking of Terms

Rank	Technical Term	Frequency
1	data	15
2	spark	6
3	hadoop	5
4	kafka	4
5	processing	3

8. Result

The experiment successfully performed **tokenization, stop word removal, and stemming** using Python. Term frequency of selected technical terms was calculated and the terms were ranked based on their frequency.

9. Conclusion

This experiment demonstrated the use of **Natural Language Processing techniques** for text preprocessing and analysis. Tokenization, stop word removal, and stemming help prepare text data for further processing. Term frequency helps identify the most significant technical terms in the document.
